# Lesson 4 — What Each Notebook Shows

Notebook guide, lesson 4 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

30 October 2026 · three notebooks

## Contents

**Notebook 01 — `01_logistic_regression_from_scratch`** **1**
Logistic regression, from scratch and from scikit-learn . . . . . 1

**Notebook 02 — `02_confusion_matrix_and_metrics`** **2**
The confusion matrix, and the metrics built on it . . . . . 2

**Notebook 03 — `03_roc_thresholds_and_imbalance`** **3**
Ranking models, and choosing a threshold on purpose . . . . . 3

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

---

## Notebook 01 — `01_logistic_regression_from_scratch`

### Logistic regression, from scratch and from scikit-learn

**What it is for.** Lesson 3 predicted a number; this one predicts a **decision**. The notebook builds the model, the cost and the gradient, then checks all of it against scikit-learn and against the coefficients that generated the data.

**The data.** 8,000 disk drives with six telemetry columns, **306 failures (3.8%)**.

**The concepts, in order.**

1. **What goes wrong with a straight line on a 0/1 label.** Fitting least squares to the label gives **3,606 of 8,000 drives — 45% of the fleet — a negative probability of failing**, and the line crosses 1 at 31.7 sectors when the worst drive observed has 28. The code runs perfectly; the output is impossible.
2. **Odds, log-odds and the sigmoid.** The table from $p = 0.01$ to $p = 0.99$ with odds and log-odds beside it, and the two lines that matter for an imbalanced problem: $\sigma(0) = 0.5$, and $\sigma(-6.2) = 0.002$ — the intercept of this fleet, so an average drive has a **0.2%** chance of failing.

3. **Why squared error is the wrong cost**, with the gradients tabulated. As the prediction slides from 0.5 to 0.001 — the model going from undecided to confidently wrong — the **log-loss gradient grows from 0.5 towards 1** while the **squared-error gradient collapses to 0.002**. Squared error stops correcting exactly when it should push hardest.
4. **Gradient descent in fifteen lines.** Log loss **0.6931 → 0.0692** over 4,000 iterations.
5. **The decision boundary**, drawn: straight, as promised, and tilted, because reallocated sectors push much harder than temperature.
6. **The truth, which only synthetic data allows.** True coefficients against from-scratch against scikit-learn, with `exp(coefficient)` as an odds multiplier: reallocated sectors 1.80 → 1.685 → 1.664 (**odds × 5.28**), spin retries 1.15 → 1.179, read errors 0.85 → 0.880, and `seek_error_rate`, generated with a coefficient of exactly **0.00**, estimated at 0.024.

**The two-minute version.** *This notebook builds logistic regression and then checks it against the truth. It starts by fitting a straight line to a 0/1 label, which runs and gives 45% of the fleet a negative probability of failing — that is why we need the sigmoid, which models the log-odds linearly and can never leave [0,1]. Then it shows why squared error is the wrong cost: when the model is confidently wrong, the squared-error gradient goes to zero and the log-loss gradient goes to one. Fifteen lines of gradient descent reproduce scikit-learn, and the last table compares both against the coefficients that generated the data — recovered closely, including the column generated with a coefficient of exactly zero, which comes back at 0.024.*

---

## Notebook 02 — `02_confusion_matrix_and_metrics`

### The confusion matrix, and the metrics built on it

**What it is for.** Notebook 1 produced a model; this one asks whether it is any good, and the first answer we reach for turns out to be the wrong one. **The lesson's number to remember is in section 1.**

**The data.** The 2,000-drive test set, 76 of them failed (3.8%).

**The concepts, in order.**

1. **The number that should worry you.**

| model | accuracy | failures caught |
|---|---|---|
| always "healthy" | **96.20%** | **0 of 76** |
| logistic regression | 97.70% | 43 of 76 |

   A model that learned something real beats one that learned nothing by **1.5 points** — and the difference that matters, 0 against 43, is nowhere in the accuracy figure.

2. **The confusion matrix: four numbers, not one.** By hand — TP 43, FP 13, FN 33, TN 1911 — and from scikit-learn, identical.

3. **Precision and recall**, each answering a different question: precision **0.7679** (of the 56 flagged, 43 really failed), recall **0.5658** (of the 76 that failed, we caught 43), F1 0.6515, specificity 0.9932.

4. **Why F1 uses the harmonic mean.** Flag every drive: precision 0.038, recall 1.000. The arithmetic mean is **0.519**, which flatters it; the harmonic mean is **0.073**, which does not.

5. **How to read `classification_report`:** go straight to the `failed` row. The `healthy` row is nearly perfect and always will be — averaging it in is how a bad model gets a respectable-looking score.

6. **The threshold is a choice.** The full sweep is the most useful table in the lesson:

| threshold | flagged | TP | FP | FN | precision | recall |
|---|---|---|---|---|---|---|
| 0.02 | 353 | 69 | 284 | 7 | 0.195 | 0.908 |
| 0.20 | 104 | 57 | 47 | 19 | 0.548 | 0.750 |
| 0.50 | 56 | 43 | 13 | 33 | 0.768 | 0.566 |
| 0.90 | 14 | 14 | 0 | 62 | 1.000 | 0.184 |

   **The fitted model is identical at every row.** Only the line between "act" and "ignore" moves.

7. **The precision-recall curve.** Average precision **0.717** against a no-skill baseline of **0.038** — and that baseline is the prevalence, which is why the curve is the honest one on imbalanced problems.

**The two-minute version.** *This is the notebook with the lesson's number in it: a model that answers "healthy" every time scores 96.20% accuracy and catches none of the 76 failures, while the real model scores 97.70% and catches 43. One and a half points separate a useful model from a worthless one, so accuracy is not the instrument. The confusion matrix splits that into four numbers, precision and recall ask different questions of them, and F1 uses the harmonic mean because a model that flags everything would otherwise score 0.52. Then the threshold sweep: the same model, unchanged, goes from catching 69 of 76 failures with 284 false alarms to catching 14 with none. The threshold is a decision, and nobody made it.*

---

## Notebook 03 — `03_roc_thresholds_and_imbalance`

### Ranking models, and choosing a threshold on purpose

**What it is for.** Notebook 2 ended on an awkward fact: every metric depended on a threshold nobody chose. This notebook builds the receiver operating characteristic (ROC) curve, shows where it misleads, and then chooses a threshold from costs.

**The concepts, in order.**

1. **The ROC curve by hand**, 2,001 points against scikit-learn's 72 (it drops points on straight segments), and the area agreeing exactly: **0.9493** both ways.

2. **What the area under the ROC curve (AUC) actually means**, verified by simulation: draw 200,000 random (failing, healthy) pairs and the failing drive scores higher in **0.9488** of them against `roc_auc_score`'s 0.9493. **AUC is the probability that a random positive outranks a random negative**, and two independent routes agree.

3. **The rare-positive trap**, which is the section to keep if only one survives:

| positive rate | AUC | avg precision | precision @0.5 |
|---|---|---|---|
| 0.0380 | 0.9677 | 0.7062 | 0.7876 |
| 0.0100 | 0.9654 | 0.4875 | 0.4619 |
| 0.0040 | 0.9735 | 0.3608 | 0.2585 |

   **AUC hardly moves** while average precision collapses from 0.71 to 0.36 — one model, drawn twice, living in different worlds depending on which curve you look at.

4. **Choosing the threshold from costs**, the ingredient no metric supplies: do nothing **197,600 €**, threshold 0.50 **87,620 €**, threshold 0.078 **45,540 €**. Choosing on purpose saves **42,080 € on 2,000 drives, 48% of the cost** — and when misses cost more than false alarms the optimum moves **down**, not up.

5. **Class weights: the same idea, applied earlier.** `class_weight="balanced"` gives recall 0.895 at threshold 0.5 and a cost of 49,500 € — and its **AUC is 0.950 against the plain model's 0.949**. Reweighting taught the model nothing new about disk failure; it moved where the 0.5 line falls, which is the same lever as the threshold.

6. **More than two classes** (optional, with slides 58–60; not needed for exercise 4). healthy / degraded / failed, and the instruction is to **read the off-diagonal cells, not the accuracy**: degraded is the hard class (recall 0.498) because it sits between two neighbours that both look like it.

**The two-minute version.** *Three things. First, the ROC curve built by hand, with AUC 0.9493 — and we verify what AUC means by drawing 200,000 random pairs of one failing and one healthy drive: the failing one scores higher 94.9% of the time. Second, where ROC misleads: make the failures rarer and AUC barely moves, from 0.968 to 0.974, while average precision falls from 0.71 to 0.36. Same model, same ranking, and on a rare problem only one of those two curves is telling you anything useful. Third, we choose the threshold from money: doing nothing costs 197,600 euros, the default threshold 87,620, and the right threshold 45,540 — 48% saved by making a decision somebody was already making by accident.*